Qualità: testing, accessibilità, prestazioni, sicurezza

Corso di Frontend Development – Modulo 06

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

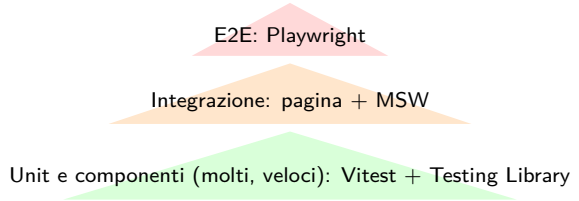
1. Qualità: testing, accessibilità, prestazioni, sicurezza
1.1 Testing
1.2 Accessibilità
1.3 Formattazione e lint
1.4 Prestazioni
1.5 Sicurezza del frontend

Sommario del modulo

1. Qualità: testing, accessibilità, prestazioni, sicurezza
1.1 Testing
1.2 Accessibilità
1.3 Formattazione e lint
1.4 Prestazioni
1.5 Sicurezza del frontend

Perché testare il frontend

- Oggi **minicommerce non ha nemmeno un test**, né un test runner configurato: partiamo da zero. È la condizione normale di un progetto giovane.
- Il frontend cambia **più spesso** del backend: ogni modifica alla UI può rompere un flusso.
- Gli agenti di programmazione scrivono codice velocissimo: i test sono il modo per **fidarsi** senza rileggere tutto.
- Regola: testate il **comportamento** che l'utente vede, non l'implementazione.



Installare Vitest nel progetto

```
cd frontend
npm install -D vitest jsdom
npm install -D @testing-library/react @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.ts: si aggiunge una sezione test a quella che c'e' gia'
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: { alias: { '@': path.resolve(import.meta.dirname, './src') } },
  test: { environment: 'jsdom', setupFiles: ['./src/test/setup.ts'], globals: true },
});

// package.json
"scripts": { "test": "vitest", "test:run": "vitest run" }
```

`npm test` resta in ascolto e riesegue solo i test toccati. L'alias `@` è già configurato: i test lo usano come il resto del codice.

Primo test: la validazione dell'articolo

```
// validateForm oggi vive DENTRO ProductFormDialog e non e' esportata: il primo
// esercizio e' estrarla in src/lib/validazione.ts, cosi' si puo' testare da sola.
// (Dopo il passaggio a react-hook-form + zod del modulo 1, l'unita' da testare
//  diventa lo schema zod: gli stessi casi, un'altra implementazione.)
import { describe, expect, it } from 'vitest';
import { EMPTY_FORM, validateForm } from './validazione';

const valido = { ...EMPTY_FORM, name: 'Mouse MX Master', categoryId: '3', price: '49.90' };

describe('validateForm', () => {
  it('accetta un articolo valido', () => {
    expect(validateForm(valido)).toEqual({});
  });
  it('rifiuta il nome vuoto', () => {
    expect(validateForm({ ...valido, name: '   ' }).name).toMatch(/obbligatorio/i);
  });
  it('rifiuta un nome oltre 32 caratteri', () => {
    expect(validateForm({ ...valido, name: 'x'.repeat(33) }).name).toMatch(/32/);
  });
  it('rifiuta un prezzo negativo', () => {
    expect(validateForm({ ...valido, price: '-1' }).price).toBeDefined();
  });
  it('vuole la categoria', () => {
    expect(validateForm({ ...valido, categoryId: '' }).categoryId).toBeDefined();
  });
});
```

Il primo test trova già un difetto

- La regola dice **massimo 32 caratteri** per il nome, ma l'input ha `maxLength={64}`: l'utente può scriverne 64 e scopre l'errore solo dopo.
- Nessuna delle due è sbagliata in assoluto: il punto è che **erano in disaccordo** e nessuno lo aveva notato.
- Scrivere il test ha costretto a leggere la regola e a decidere. Capita quasi sempre.
- Altre cose da estrarre e testare subito: `getErrorMessage` e `formatPrice` in `lib/api.ts`, pure e già isolate.
- `formatPrice(49.9)` su locale `it-IT` dà `49,90 €`: un test lo blocca prima che qualcuno "semplifichi" in dollari.

Testing Library: il form come lo usa un utente

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { ProductFormDialog } from './ProductFormDialog';

it('segnala il nome troppo lungo e non salva', async () => {
  const onSaved = vi.fn();
  render(<ProductFormDialog open product={null} onOpenChange={vi.fn()} onSaved={onSaved} />);

  await userEvent.type(screen.getByLabelText(/nome/i), 'T'.repeat(33));
  // qui solo il campo Nome e' "touched": c'e' un solo alert nella pagina
  expect(await screen.findByRole('alert')).toHaveTextContent(/32 caratteri/);

  await userEvent.click(screen.getByRole('button', { name: /salva/i }));
  expect(onSaved).not.toHaveBeenCalled();
});
```

- Si cerca per **ruolo e testo** (`getByRole`, `getByLabelText`), mai per classe CSS.
- Il dialog chiama `fetchCategories()` all'apertura: senza una finta API il test fallisce con un errore di rete. Serve MSW.

MSW: una finta API che parla come la vera

```ts
// src/mocks/handlers.ts
import { http, HttpResponse } from 'msw';

const CATEGORIE = [{ id: 1, name: 'Mouse', description: '', productCount: 1, createdAt: '2026-01-01' }];
const MOUSE = {
  id: 7, name: 'Mouse MX Master', description: 'Wireless', categoryId: 1, category: 'Mouse',
  brand: 'Logitech', price: 49.9, stock: 3, imageUrl: null, createdAt: '2026-01-01',
};

export const handlers = [
  http.get('/api/categories', () => HttpResponse.json(CATEGORIE)),
  http.get('/api/products', ({ request }) => {
    const search = new URL(request.url).searchParams.get('search') ?? '';
    const righe = [MOUSE].filter((p) => p.name.toLowerCase().includes(search.toLowerCase()));
    return HttpResponse.json(righe);   // la vera API risponde con un array, non un oggetto
  }),
];

// src/test/setup.ts
import '@testing-library/jest-dom/vitest';
import { server } from '../mocks/server';
beforeAll(() => server.listen()); afterEach(() => server.resetHandlers()); afterAll(() => server.close());
```

Gli stati di CatalogPage

```tsx
// src/test/render.tsx -- router in memoria + AuthProvider, nuovi per ogni test
export function renderApp(path: string) {
  return render(
    <MemoryRouter initialEntries={[path]}><AuthProvider><App /></AuthProvider></MemoryRouter>,
  );
}

it('caricamento, poi dati', async () => {
  renderApp('/');
  expect(screen.getByText(/caricamento/i)).toBeInTheDocument();
  expect(await screen.findByText('Mouse MX Master')).toBeInTheDocument();
});

it('stato vuoto quando il filtro non trova niente', async () => {
  renderApp('/');
  await userEvent.type(screen.getByPlaceholderText(/cerca/i), 'zzz');
  expect(await screen.findByText(/nessun articolo trovato/i)).toBeInTheDocument();
});
```

Il quarto stato, l'**errore**, oggi non è testabile: `CatalogPage` lo mostra solo con un toast e la lista cade nello stato vuoto, indistinguibile da "nessun risultato". È il difetto che il modulo 1 corregge con uno stato di errore vero.

Playwright: il flusso completo nel browser vero

```
npm init playwright@latest   # installa i browser, crea playwright.config.ts e tests/
```

```
import { expect, test } from '@playwright/test';

test('l amministratore crea un articolo e lo trova in catalogo', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('admin@minicommerce.dev');
  await page.getByLabel('Password').fill('admin123');
  await page.getByRole('button', { name: 'Accedi' }).click();
  await expect(page).toHaveURL('/admin');

  await page.getByRole('button', { name: 'Nuovo articolo' }).click();
  await page.getByLabel('Nome *').fill('Mouse di prova');
  await page.getByLabel('Categoria *').click();
  await page.getByRole('option', { name: 'Mouse' }).click();
  await page.getByLabel(/Prezzo/).fill('19.90');
  await page.getByRole('button', { name: 'Salva' }).click();

  await page.getByRole('link', { name: 'Catalogo' }).click();
  await expect(page.getByText('Mouse di prova')).toBeVisible();
});
```

E2E: l'eliminazione, e un test verde che inganna

```
test('e poi lo elimina', async ({ page }) => {
  // window.confirm non e' un elemento della pagina: Playwright lo gestisce a parte
  page.on('dialog', (d) => d.accept());
  await page.goto('/admin');

  await page.getByRole('row', { name: /Mouse di prova/ })
    .getByRole('button', { name: 'Elimina' }).click();

  await expect(page.getByText('Mouse di prova')).toHaveCount(0);
});
```

- Questo test **passa**, e proprio per questo è istruttivo: i pulsanti della tabella hanno solo `title="Elimina"`, e `title` entra nel calcolo del nome accessibile come **ultima risorsa**. `getByRole` lo trova.
- Ma quel nome è un incidente, non una scelta: `title` non compare mai su telefono e diversi screen reader sono configurati per ignorarlo.
- Morale: – **un test verde non certifica l'accessibilità**, e nemmeno axe segnala questo caso. Si vede solo usando l'app, ed è la prossima sezione.

E2E: il cliente, e il carrello vietato agli amministratori

```
test('un cliente si registra e aggiunge al carrello', async ({ page }) => {
  await page.goto('/login');
  await page.getByRole('button', { name: 'Crea account' }).click();   // il tasto che cambia modo
  await page.getByLabel('Nome').fill('Cliente Prova');
  await page.getByLabel('Email').fill(`c${Date.now()}@test.dev`);  // il seed ha solo l admin
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Crea account cliente' }).click();
  await page.getByRole('button', { name: 'Aggiungi al carrello' }).first().click();
  await page.getByRole('link', { name: /Carrello/ }).click();
  await expect(page.getByText('Totale')).toBeVisible();
});

test('a un amministratore il carrello e vietato', async ({ request }) => {
  const ADMIN = { email: 'admin@minicommerce.dev', password: 'admin123' };  // dal seed
  const { token } = await (await request.post('/api/auth/login', { data: ADMIN })).json();
  const r = await request.get('/api/cart', { headers: { Authorization: `Bearer ${token}` } });
  expect(r.status()).toBe(403);   // non 401: autenticato, ma non autorizzato
});
```

- Il carrello vive **sul server**, legato all'utente: questo test passa dal database vero, non da MSW. Ogni esecuzione crea un cliente nuovo, perché il seed inserisce solo l'amministratore.
- La UI rimanda l'admin su `/admin`, ma il divieto vero è `requireCustomer` nel backend: **autenticazione e autorizzazione sono due cose diverse**, e il test lo verifica dove conta.

Cosa testare (e cosa no)

Sì

- Logica pura: `validateForm`, `formatPrice`, `getErrorMessage`
- Componenti con comportamento: il form articolo, i filtri del catalogo
- Pagine: i 4 stati (caricamento, errore, vuoto, dati)
- I flussi veri end-to-end: login, CRUD articolo, registrazione, carrello e ordine
- Le regole di autorizzazione: il carrello risponde 403 a un admin
- Ogni bug corretto: prima il test che fallisce, poi la fix

No (o con cautela)

- Snapshot di interi componenti: si rompono a ogni ritocco e nessuno li legge
- Dettagli di implementazione (stato interno, chiamate a hook)
- I componenti di `ui/`: vengono da shadcn e Radix, già testati a monte
- Stile e layout, se non con test visivi dedicati

Accessibilità: non è opzionale

- Il 15–20% delle persone ha una disabilità permanente; tutti ne hanno di temporanee (braccio rotto, sole sullo schermo).
- **European Accessibility Act**: in vigore da giugno 2025 per e-commerce, banche, trasporti, e-book, servizi digitali. minicommerce è un e-commerce: ci rientrerebbe.
- In Italia si aggiunge la **Legge Stanca** per PA e grandi aziende.
- Standard di riferimento: **WCAG 2.2**, livello AA. Quattro principi: Percepibile, Utilizzabile, Comprensibile, Robusto.
- HTML semantico fatto bene copre l'80% del lavoro; i `<div onClick>` lo distruggono.

minicommerce: cosa è già fatto bene

- + Ogni campo ha un `<Label htmlFor>` collegato al suo `id`, nel login e nel form articolo. È la cosa che più spesso manca, e qui c'è.
- + `aria-invalid` sui campi non validi, e sul nome anche `aria-describedby` con `role="alert"` sul messaggio: lo screen reader lo annuncia da solo.
- + L'anteprima immagine ha un `alt` descrittivo; l'input file è `sr-only` ma pilotato da un `<label>` visibile e raggiungibile con Tab.
- + `autoComplete="email"` e `"current-password"` nel login: i gestori di password funzionano.
- + `Dialog` e `Select` vengono da Radix: trappola del focus, Esc, ruoli ARIA e frecce sono già corretti. **Usare una libreria serie è una scelta di accessibilità.**

...e cosa va corretto

```
// 1. Icona sola: title e' un nome di ultima risorsa. AdminPage (modifica,
//    elimina) e CatalogPage (direzione ordinamento). Il carrello fa gia' bene.
<Button size="icon" title="Elimina"><Trash2 /></Button>                    // NO
<Button size="icon" aria-label={`Elimina ${product.name}`}>                // SI
  <Trash2 aria-hidden />
</Button>

// 2. aria-describedby c'e' solo sul campo Nome: gli altri errori non sono collegati
<Input id="p-price" aria-invalid={!!err} aria-describedby={err && 'p-price-error'} />
<p id="p-price-error" role="alert">{err}</p>

// 3. index.html dichiara lang="en" su un'app interamente in italiano
<html lang="it">

// 4. CatalogPage: la Card porta all'articolo con onClick su un <div>:
//    niente Tab, niente Invio, niente annuncio. Serve un vero link.
<Card onClick={() => navigate(`/article/${product.id}`)}>                  // NO
<Link to={`/article/${product.id}`}>{product.name}</Link>                  // SI
```

`title` compare come tooltip solo col mouse, dopo un'attesa, e non esiste su telefono: formalmente conta come nome, in pratica non lo è.

Tastiera e focus

- Tutto ciò che si fa con il mouse si deve poter fare con **Tab, Invio, Spazio, frecce, Esc**. Provate 10 minuti di minicommerce senza toccare il mouse.
- `window.confirm` in `AdminPage`: sposta il focus fuori dalla pagina, non si può stilare né annunciare. Sostituitelo con l'**AlertDialog** deciso nel modulo 1.
- L'anello di focus deve essere **visibile**: mai `outline: none` senza alternativa. I componenti shadcn lo gestiscono: non toglietelo.
- SPA: passando da `/` a `/admin` il focus resta dov'era e lo screen reader non dice niente. Spostatelo sull'`<h1>` della nuova pagina.
- Link "salta al contenuto" come primo elemento, per non ritabulare l'`Header` ogni volta.

Annunciare i risultati del filtro

```
// Nel catalogo il numero di risultati cambia mentre si digita: chi non vede
// lo schermo non lo sa. Un contatore in una regione live risolve.
<p aria-live="polite" role="status" className="text-sm text-muted-foreground">
  {loading ? 'Ricerca in corso...' : `${products.length} articoli trovati`}
</p>
```

- `aria-live="polite"` annuncia quando l'utente fa una pausa; `assertive` interrompe: solo per errori gravi.
- La regione deve **esistere già** nel DOM quando il testo cambia: se la create insieme al messaggio, spesso non viene annunciata.
- I toast di `sonner` sono già regioni live, ma sono **effimeri**: un errore che l'utente deve poter rileggere non va in un toast.

Colori, movimento, testo

- Contrasto minimo **4,5:1** per il testo normale, 3:1 per testo grande e componenti UI.
- Da misurare in minicommerce: il Badge "Esaurito" è `text-destructive` su `bg-destructive/10`, cioè rosso su rosa pallido. Con ogni probabilità non arriva a 4,5:1.
- Mai il colore come *unica* informazione. Qui va bene: "Esaurito" è anche una parola, non solo un badge rosso.
- `prefers-reduced-motion`: disattivare le animazioni non essenziali.
- Testo ridimensionabile al 200% senza rotture: la barra dei filtri ha larghezze fisse (`w-28`, `w-44`) da controllare.

Strumenti

```
npm install -D eslint-plugin-jsx-a11y  # errori a11y mentre scrivete (oxlint ha regole simili)
npm install -D vitest-axe              # regole axe nei test dei componenti
```

```
import { axe } from 'vitest-axe';
it('il form articolo non ha violazioni a11y', async () => {
  const { container } = render(<ProductFormDialog open product={null} ... />);
  expect(await axe(container)).toHaveNoViolations();
});
```

- Estensione **axe DevTools** e scheda Accessibility di Lighthouse: trovano il 30–40% dei problemi, fra cui subito `lang="en"` e i contrasti.
- Il resto si trova **usando** l'app: solo tastiera, poi con **NVDA** (Windows, gratuito) o VoiceOver.
- In E2E: `@axe-core/playwright` per l'audit di ogni pagina.

Un formattatore non è una questione di gusto

```
npm run lint                # oxlint: velocissimo, trova errori ma NON formatta
npm install -D prettier     # oppure: npm install -D --save-exact @biomejs/biome
npx prettier --write "src/**/*.{ts,tsx,css}"
```

- Il progetto usa **oxlint** e non ha Prettier. Risultato: `AGENTS.md` chiede virgolette doppie nel frontend, ma le seguono solo `CatalogPage`, `ArticlePage`, `ProductFormDialog` e `components/ui/`; `App.tsx`, `Header.tsx`, `AddToCartButton`, `lib/` e le altre pagine usano le singole.
- Il costo non è estetico: sono **righe che cambiano senza motivo** nei diff, e conflitti su codice identico.
- Scegliete *uno* strumento, eseguitelo una volta su tutto (un commit solo, "formattazione") e poi bloccatelo con un hook pre-commit: `husky` e `lint-staged`, oppure i git hooks nativi.
- Biome formatta *e* lint in un solo strumento; Prettier formatta e si affianca a oxlint. Entrambe difendibili: la scelta peggiore è non scegliere.

Il problema numero uno: le immagini in base64

```
// ProductFormDialog: la foto dell'articolo diventa una data URL...
const MAX_IMAGE_SIZE = 5 * 1024 * 1024;      // fino a 5 MB per articolo
reader.readAsDataURL(file);                   // "..."

// ...finisce in una colonna TEXT senza limiti...
imageUrl: text('image_url'),

// ...e torna indietro DENTRO OGNI RIGA di GET /api/products
const productSelection = { /* ... */ imageUrl: products.imageUrl };
```

- Base64 aggiunge un terzo: i 16 articoli del seed con una foto da 2 MB ciascuno fanno **oltre 40 MB di JSON** a ogni apertura del catalogo.
- E il catalogo **non mostra nemmeno l'immagine**: la paghiamo senza usarla. La mostrano `ArticlePage` e `CartPage`, che però hanno bisogno di un articolo solo.
- Nessuna cache possibile: il JSON cambia a ogni filtro, il browser riscarica tutto.

Lo stesso difetto è anche un bug

```
// backend/src/server.ts
app.use(express.json());   // limite predefinito del corpo: 100 KB
```

- Il client accetta 5 MB, il server ne accetta 100 KB: una foto normale fa fallire il salvataggio, e l'utente vede un errore generico perché il gestore centralizzato risponde 500.
- – Soluzione sbagliata: `express.json({ limit: '10mb' })`. Sposta il problema e peggiora quello di prestazioni.
- + Soluzione giusta (è il modulo 3): **upload multipart**, il file su disco o su storage, nel database solo il percorso. Poi `<img loading="lazy">` con `width` e `height` e immagini ridimensionate.
- Da aggiungere lato server: controllo del tipo reale del file e della dimensione. La validazione nel browser aiuta l'utente, non difende l'API.

Il secondo problema: nessuna paginazione

```
// backend: GET /api/products fa .all(), cioe' tutto il catalogo, sempre
const rows = selectProducts().where(...).orderBy(...).all();

// Da aggiungere all'API (esercizio): ?limit=24&offset=0
// risposta: { items: Product[], total: number }
// nel frontend bastano i filtri in piu': l'hook rifa' la richiesta da solo
const { data, loading } = useProducts({ ...filtri, limit: 24, offset: pagina * 24 });
```

- Con i 16 articoli del seed non si vede niente. Inserite 5.000 righe nel seed e riprovate: il server ordina tutto, la rete trasporta tutto, React monta 5.000 `Card`.
- Oltre le 200 righe visibili servono **virtualizzazione** (per esempio TanStack Virtual) o paginazione. Meglio la paginazione: risolve anche la rete.

Misurare prima di ottimizzare

- **Lighthouse** (in DevTools, con throttling mobile): punto di partenza, ripetibile.
- **Network** di DevTools: la prima cosa da guardare è il peso della risposta di `/api/products`.
- **Performance panel**: dove va il tempo (script, layout, paint); i *long task* sopra 50 ms.
- **React DevTools Profiler**: quali componenti si ri-renderizzano e perché.
- `npm run build` con `rollup-plugin-visualizer`: cosa c'è nel bundle e quanto pesa.

Budget di prestazioni per minicommerce

JS iniziale < 200 KB compressi; risposta di `/api/products` < 100 KB; LCP $< 2{,}5$ s su 4G; INP < 200 ms.

Le ottimizzazioni che contano davvero

1. **Meno dati**: paginazione, nessun base64 nelle risposte, solo i campi che servono.
2. **Meno JavaScript**: code splitting per rotta (il modulo 1 lo fa su `/admin`), dipendenze leggere, controllate su bundlephobia.
3. **Immagini**: formato moderno, dimensioni giuste, `loading="lazy"`, `width` e `height` sempre per non far saltare il layout (CLS).
4. **Font**: un solo file, `woff2`, `font-display: swap`, subset dei caratteri.
5. **Cache**: asset con hash e `immutable`; un hook che non rifa' la richiesta se i filtri non cambiano, al posto del debounce con `setTimeout` fatto a mano.
6. **Rete**: HTTP/2 o 3, CDN, compressione Brotli, preconnect all'API.

Quando il filtro diventa locale

```
// Col catalogo offline della PWA (modulo 3) il filtro gira nel browser:
// su liste grandi la digitazione comincia a scattare.
const [q, setQ] = useState('');
const qDiff = useDeferredValue(q);            // React aggiorna la lista "con calma"
const visibili = useMemo(() => filtra(articoli, qDiff), [articoli, qDiff]);

// Lavoro pesante davvero (l'esportazione CSV del modulo 2): fuori dal main thread
const worker = new Worker(new URL('./csv.worker.ts', import.meta.url), { type: 'module' });
worker.postMessage(articoli);
worker.onmessage = (e) => scarica(e.data);
```

- Vite gestisce i worker come moduli: TypeScript e import funzionano anche lì.
- Perdite di memoria tipiche: `setTimeout` e listener senza pulizia negli effetti. Il debounce di `CatalogPage` lo fa bene (`return () => clearTimeout(timer)`): copiate quel pattern.

Il modello mentale

- Il codice frontend gira **sulla macchina dell'utente**: chiunque può leggerlo, modificarlo, chiamare l'API a mano.
- Quindi: **nessun segreto nel client**, **nessun controllo di sicurezza solo nel client**. L'API valida e autorizza sempre.
- minicommerce su questo è corretto: `AdminPage` nasconde la UI, ma è `requireAdmin` nel backend che blocca davvero POST, PUT e DELETE; e il carrello è chiuso dietro `requireCustomer`, che risponde **403** a un amministratore.
- Il frontend protegge *l'utente* (da script malevoli, da siti che lo impersonano), non l'API.
- Le minacce principali: **XSS**, furto di sessione, CSRF, dipendenze compromesse, configurazioni sbagliate.

XSS: come nasce e come si evita

```
// React fa l'escape di tutto cio' che interpola: questo e' SICURO
<p>{product.description}</p>   // "<img onerror=...>" viene mostrato come testo

// Questo NO: HTML grezzo dal server o dall'utente
<div dangerouslySetInnerHTML={{ __html: product.descriptionHtml }} />

// Se dovete davvero renderizzare HTML (descrizioni in markdown), sanificate
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }} />

// Altro ingresso da sorvegliare: imageUrl arriva dal database, non da voi
<img src={product.imageUrl} />   // "javascript:..." o un URL di tracciamento esterno
```

- Oggi minicommerce non usa `dangerouslySetInnerHTML`: +. Mantenete l'abitudine.
- Un XSS riuscito legge `localStorage` (il token!) e fa richieste come l'utente. Difesa in profondità: **Content Security Policy**.

Content Security Policy

```
# header HTTP inviato dal server che serve la SPA (o <meta http-equiv> in index.html)
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
  connect-src 'self' https://api.minicommerce.example;
  frame-ancestors 'none';
  base-uri 'self'; form-action 'self'
```

- minicommerce oggi **non ha nessuna CSP**: né header, né meta tag in `index.html`.
- Anche se un attaccante inietta `<script>`, con la CSP il browser **non lo esegue**.
- `connect-src` limita a chi il JS può parlare: un token rubato non può essere spedito altrove.
- `img-src data:` serve finché le immagini sono base64: togliendo il base64 la CSP diventa più stretta. Partite in `Report-Only` e guardate cosa rompe.

Sessione: la versione da produzione

- Oggi: JWT da 8 ore in `localStorage`, aggiunto dall'interceptor di richiesta di axios. Semplice, e vulnerabile a qualunque XSS.
- Per il **web puro**: token in cookie `httpOnly; Secure; SameSite=Lax` impostato dall'API. Il JS non lo vede: un XSS non può rubarlo.
- Con i cookie serve difendersi da **CSRF**: `SameSite` già aiuta; per le POST da altri siti l'API richiede un header custom o un token anti-CSRF.
- Il frontend cambia poco: `withCredentials: true` su axios e via l'interceptor del token.
- Per **desktop e mobile**: storage sicuro del sistema (`safeStorage`, Keychain, Preferences), come nei moduli 2 e 3.
- Token brevi (15 min) con refresh: un furto vale poco e per poco. Otto ore sono tante.

CORS, spiegato una volta per tutte

- CORS è una **protezione del browser per l'utente**: impedisce a `sito-cattivo.com` di leggere risposte della vostra API usando i cookie dell'utente.
- **Non** protegge l'API da chiamate dirette: curl, Postman e un'app nativa non hanno CORS.
- L'API dichiara chi può leggerla: `Access-Control-Allow-Origin: https://minicommerce.example`. Mai `*` insieme a credenziali.
- Le richieste "non semplici" (JSON, header custom) fanno prima una `OPTIONS` di preflight: l'API deve risponderle.
- In sviluppo il problema non si vede, perché Vite fa da **proxy**: `/api` è la stessa origine. In produzione i domini sono diversi e CORS torna a contare.

I dati che non volete nel vostro sistema

```
// CartPage: il form del pagamento dimostrativo chiede numero, scadenza e codice...
await placeMockOrder({
  paymentCardholder: cardholder,
  paymentLastFour: cardNumber.slice(-4),     // ...ma al server parte solo questo
});
// backend/src/routes/orders.ts: l'ordine registra solo cio' che e' arrivato
paymentCardholder: paymentCardholder.trim(),
paymentLastFour: paymentLastFour.trim().slice(-4),
```

- `POST /api/orders` registra l'ordine con stato `accepted` e svuota il carrello: nessun circuito di pagamento, nessuna carta validata, nessuna scorta scalata. È una **simulazione**, e la pagina lo dice.
- Numero completo, scadenza e codice di sicurezza **non lasciano mai il browser**: nessuna richiesta li contiene, nessun log li registra, nessuna colonna li conserva. +
- Chi tratta davvero i dati di una carta entra nel perimetro **PCI DSS**: audit, segmentazione di rete, responsabilità legale. Un e-commerce vero ci resta fuori: il pagamento lo gestisce un fornitore (Stripe, PayPal), voi tenete un identificativo.
- **Il dato più sicuro è quello che non raccogliete.**

I difetti reali, nel repository

```
// backend/src/middleware/auth.ts
export const JWT_SECRET = process.env.JWT_SECRET ?? 'dev-secret-change-me';
// Se in produzione la variabile manca, l'app parte comunque: chiunque conosca
// questa riga si firma un token con isAdmin true. Nessun errore, nessun log.

// backend/src/server.ts
app.use(cors());          // Access-Control-Allow-Origin: * per qualunque sito
app.use(express.json());  // nessun helmet, nessuna CSP, nessun header di sicurezza

// backend/src/routes/auth.ts: login e register senza alcun limite ai tentativi.
// "admin123" si trova con qualche migliaio di richieste in un minuto, e la
// registrazione lascia creare account a volonta', senza verifica dell'email.
```

Non sono ipotesi: sono righe che potete aprire e leggere. **Un audit comincia così**, non da uno strumento.

Le correzioni

```
// 1. Nessun segreto con valore di default
if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET mancante');
export const JWT_SECRET = process.env.JWT_SECRET;

// 2. CORS con origini esplicite, prese dalla configurazione
app.use(cors({ origin: process.env.CORS_ORIGIN?.split(',') ?? [], credentials: true }));

// 3. Header di sicurezza e corpo limitato
app.use(helmet());
app.use(express.json({ limit: '1mb' }));

// 4. Limite ai tentativi su login e registrazione
import rateLimit from 'express-rate-limit';
app.use(['/api/auth/login', '/api/auth/register'], rateLimit({ windowMs: 60000, limit: 10 }));
```

E nel seed: la password dell'amministratore da variabile d'ambiente, non `admin123` scritto nel codice.

Dati e script distruttivi

- \+ Il database è già fuori dal repository: `*.db`, `*.db-shm`, `*.db-wal` e `.env` sono nel `.gitignore`. Verificatelo con `git check-ignore`, non a memoria: un database committato è una fuga di dati permanente.
- − `db:push` usa `-force`: applica anche le modifiche che perdono dati, senza chiedere. Comodo in sviluppo, **inaccettabile** su un database vero.
- − `db:seed` comincia svuotando *tutte* le tabelle, utenti, carrelli e ordini compresi; `db:reset` cancella il file. Nomi innocui, effetti irreversibili.
- In produzione servono **migrazioni** versionate (`drizzle-kit generate` e `migrate`), non `push`. E backup.
- Regola pratica: uno script che distrugge dati va chiuso dietro una variabile d'ambiente, o rinominato in modo che faccia paura.

Dipendenze e catena di fornitura

```
npm audit                          # vulnerabilita' note nelle dipendenze
npm outdated                       # cosa e' vecchio
npm ci    # in CI: esattamente le versioni del lockfile
```

- Ogni pacchetto npm è codice che gira nel vostro build e nel browser dell'utente. Nel 2024–2025 diversi pacchetti popolari sono stati compromessi.
- Guardate le dipendenze di minicommerce: `shadcn` è uno strumento da riga di comando e sta fra le *dependencies* del frontend. Meno pacchetti, meno superficie.
- Regole: **lockfile committato**, aggiornamenti automatici con revisione (Renovate o Dependabot), pacchetti mantenuti e usati da molti.
- `npm install` esegue gli script di installazione dei pacchetti (`postinstall` e simili): è codice arbitrario che gira sul vostro PC. `npm install --ignore-scripts`, o `ignore-scripts=true` nel file `.npmrc` del progetto, lo impedisce.

Checklist di sicurezza per minicommerce

- ☐ `JWT_SECRET` obbligatorio, nessun default
- ☐ CORS con origini esplicite da configurazione
- ☐ `helmet()` e CSP almeno `script-src 'self'`
- ☐ Limite ai tentativi su login e registrazione
- ☐ Corpo limitato, upload validato lato server
- ☐ Autorizzazione per ruolo sul server (`requireAdmin`, `requireCustomer`), non solo UI nascosta
- ☐ Token in cookie httpOnly sul web; storage sicuro su desktop e mobile
- ☐ Token brevi con refresh; logout che invalida
- ☐ Password admin da variabile d'ambiente
- ☐ Migrazioni al posto di `push --force`
- ☐ Nessun dato di carta nel sistema: solo intestatario e ultime quattro cifre
- ☐ `npm audit` in CI, lockfile bloccato, HTTPS ovunque

Riferimento: OWASP Top 10 e i Cheat Sheet OWASP "HTML5 Security" e "Content Security Policy".

Laboratorio 6

Laboratorio: Portare minicommerce a una qualità professionale

1. Test: Vitest; estraete e coprite `validateForm`; form articolo e `CatalogPage` con MSW; un E2E lato admin e uno lato cliente (registrazione, carrello, ordine), più il 403 del carrello.
2. Accessibilità: zero violazioni axe su tutte le pagine; `aria-label` su ogni pulsante con sola icona; `lang="it"`; contrasto di "Esaurito" misurato; 10 minuti di sola tastiera.
3. Prestazioni: peso di `/api/products` prima e dopo aver tolto il base64; Lighthouse mobile ≥ 90; bundle iniziale sotto 200 KB.
4. Sicurezza: chiudete almeno **quattro** voci della checklist, un commit per voce.
5. Formattazione: Prettier o Biome, un commit solo di riformattazione, un hook pre-commit.

Domande di verifica

Perché `title` non basta su un pulsante con sola icona? Cosa blocca una CSP con `script-src 'self'`? CORS protegge l'API o l'utente? Perché il base64 peggiora *ogni* richiesta al catalogo? Perché il carrello risponde 403 e non 401 a un admin? Perché il numero della carta non arriva al backend?